



EDITORA AGÊNTICA

CODE REVIEW GRAPH: FUNDAMENTOS

Heverton Eduardo Peres

ESPECIALISTA EM MARKETING E DESENVOLVIMENTO DE SOLUÇÕES

Heverton Eduardo Peres

Fundamentos

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

Code Review Graph: Fundamentos	4
Capítulo 1: Por Que Jogar o Código Inteiro no LLM Não Funciona	5
O Problema Que Todo Desenvolvedor Já Enfrentou	5
O Custo Real dos Tokens	5
O Blast Radius: Quanto do Seu Código Realmente Importa?	6
Por Que Grafos São a Resposta	6
A Mecânica da Compressão	6
O Pipeline Completo	7
O Código por Trás da Magia	7
Caso Real: Startup de Fintech	9
Armadilhas a Evitar	9
Capítulo 2: Instalação e Configuração	11
De Zero a Review Automática em 10 Minutos	11
Arquitetura em Quatro Componentes	11
Dependências	11
Instalação via Pip	12
Configuração Manual via .mcp.json	13
Git Hooks: Reviews Automáticas	14
Watch Mode: Feedback em Tempo Real	14
Construção do Grafo Inicial	15
Integração com CI/CD	15
Verificação e Diagnóstico	16
Casos Reais	17
Erros Comuns na Instalação	17
Próximos Passos	18

Code Review Graph: Fundamentos

Ebook derivado de Code Review Graph: Redução de Contexto para Revisão de Código com IA

Heverton Eduardo Peres

Capítulo 1: Por Que Jogar o Código Inteiro no LLM Não Funciona

O Problema Que Todo Desenvolvedor Já Enfrentou

Você já tentou mandar um repositório inteiro para um LLM revisar e viu o custo explodir? Se sim, este ebook é para você.

Code reviews assistidas por IA se tornaram essenciais. Mas a abordagem mais óbvia — jogar todo o código no contexto do modelo — gera custos proibitivos e resultados genéricos demais para serem úteis.

Vamos aos números. O Flask, um framework relativamente pequeno, tem 143.594 tokens de código fonte. Ler tudo isso custa entre USD 4,20 e USD 21,00 por revisão, dependendo do modelo. E projetos maiores? Chromium, Linux kernel — nem os modelos com janela de 1 milhão de tokens conseguem processar tudo de forma significativa.

O conceito-chave que resolve isso se chama **blast radius** — o raio de impacto semântico de uma alteração no código. E a ferramenta que permite mapeá-lo são os **grafos de dependência**.

Com o Code Review Graph, a redução mediana de tokens é de **65x** em relação à leitura integral, sem perder cobertura semântica.

O Custo Real dos Tokens

Cada token custa dinheiro. Um token representa aproximadamente 4 caracteres em inglês ou 2 em português. O custo varia por modelo, mas o padrão é claro: mais tokens, mais dinheiro.

Considere um pull request com 15 arquivos modificados num repositório de 500 arquivos e 120.000 linhas. O contexto completo pode chegar a 800.000 tokens — USD 24,00 só para a entrada, sem contar a resposta.

Mas o pior não é o custo. É a qualidade. Quando o modelo recebe código demais, ele se perde. Gera reviews superficiais, genéricas, que não capturam bugs reais. É como pedir a alguém para ler um romance inteiro e apontar apenas os erros de digitação — no final, ele vai apontar coisas óbvias e perder o que importa.

O Blast Radius: Quanto do Seu Código Realmente Importa?

Quando você modifica uma função que é chamada por 47 outras funções em 12 arquivos, o impacto da sua alteração se espalha por todos esses 12 arquivos. Esse é o blast radius.

Em código com alto acoplamento, mudanças pequenas propagam efeitos por todo o sistema. Em código com alta coesão, alterações ficam contidas em módulos bem definidos.

Para code review com IA, o blast radius determina o contexto mínimo necessário. Se o modelo vê apenas o diff, ele não avalia impactos. Se vê o sistema inteiro, o custo é impossível. A solução: mapear o blast radius real e enviar apenas o contexto semântico relevante.

Por Que Grafos São a Resposta

Um grafo de dependências é uma representação onde **nós** são elementos do código (funções, classes, arquivos) e **arestas** são as relações entre eles (chamadas, importações, herança).

A vantagem do grafo é transformar a code review de um problema de “entender tudo” em um problema de “navegar no grafo certo”. O grafo faz a triagem estrutural; o modelo faz a análise semântica.

No caso do Flask: uma alteração em `app.py` (profundidade 2) inclui apenas 38 arquivos com 8.723 tokens — redução de 16x. Uma alteração em `contrib/debug.py` inclui apenas 3 arquivos com 412 tokens — redução de 349x.

A Mecânica da Compressão

A compressão semântica não é redução aleatória. Ela é guiada por quatro princípios:

Nós centrais vs. periféricos. Funções utilitárias e interfaces públicas têm blast radius grande e sempre entram no contexto. Funções auxiliares e testes periféricos ficam de fora.

Profundidade controlada. A BFS (Breadth-First Search) a partir dos arquivos alterados define “níveis de impacto”. Nível 0: arquivos modificados. Nível 1: quem chama/importa. Nível

2: quem interage com o nível 1. A configuração padrão (profundidade 2) captura 95% dos bugs reais.

Filtragem por tipo. Uma chamada de função é mais importante que uma importação, que é mais importante que uma referência em comentário. O grafo pondera as arestas por tipo.

Deduplicação e sumarização. Arquivos grandes são comprimidos: assinaturas e docstrings no lugar de corpos inteiros. Apenas código diretamente relevante entra integralmente.

O Pipeline Completo

O fluxo funciona assim:

1. O diff do pull request é analisado por um parser de AST
2. O grafo de dependências é construído ou atualizado
3. A BFS calcula o blast-radius a partir dos arquivos modificados
4. Arestas são filtradas por tipo e peso
5. O contexto comprimido é gerado
6. O LLM recebe apenas o necessário e gera comentários estruturados

Comparação de abordagens:

Abordagem	Tokens	Custo (USD)	Cobertura	Qualidade
Leitura integral	143.594	4,20–21,00	100% (ruidosa)	Baixa
Apenas diff	2.340	0,07–0,35	15%	Média
Code Review Graph	2.209	0,06–0,33	92%	Alta

O Código por Trás da Magia

A seguir, a implementação básica do grafo de dependências em Python:

```
from dataclasses import dataclass, field
from typing import Dict, List, Set
from collections import defaultdict
import ast, os

@dataclass
class DependencyEdge:
```

```

source: str
target: str
edge_type: str # 'call', 'import', 'inherit', 'use_data'
weight: float = 1.0

@dataclass
class CodeGraph:
    nodes: Set[str] = field(default_factory=set)
    edges: List[DependencyEdge] = field(default_factory=list)
    adjacency: Dict[str, List[DependencyEdge]] = field(
        default_factory=lambda: defaultdict(list)
    )

    def add_node(self, node_id: str) -> None:
        self.nodes.add(node_id)

    def add_edge(self, edge: DependencyEdge) -> None:
        self.edges.append(edge)
        self.adjacency[edge.source].append(edge)
        self.add_node(edge.source)
        self.add_node(edge.target)

    def blast_radius(self, changed_files: List[str], depth: int = 2) ->
Set[str]:
        visited: Set[str] = set()
        frontier = set(changed_files)
        for _ in range(depth):
            next_frontier: Set[str] = set()
            for node in frontier:
                if node in visited:
                    continue
                visited.add(node)
                for edge in self.adjacency.get(node, []):
                    if edge.target not in visited:
                        next_frontier.add(edge.target)
            frontier = next_frontier
        return visited

```

O blast radius com filtragem e peso acumulado:

```

def blast_radius_with_filter(
    graph: CodeGraph,
    changed_files: List[str],
    depth: int = 2,
    min_weight: float = 0.3,
) -> Dict[str, float]:
    scores: Dict[str, float] = {}

```



```

frontier = {f: 1.0 for f in changed_files}

for level in range(depth):
    next_frontier: Dict[str, float] = {}
    for node, current_score in frontier.items():
        if node in scores:
            continue
        scores[node] = current_score
        for edge in graph.adjacency.get(node, []):
            if edge.target in scores or edge.weight < min_weight:
                continue
            propagated = current_score * edge.weight * 0.7
            next_frontier[edge.target] = max(
                next_frontier.get(edge.target, 0), propagated
            )
        frontier = next_frontier
    return scores

```

O fator de decaimento 0.7 garante que nós distantes recebam scores menores. A filtragem por `min_weight` exclui relações triviais.

Caso Real: Startup de Fintech

Uma startup com repositório monolítico (2.340 arquivos, 890.000 linhas) testou três abordagens:

- **Apenas diff:** custo USD 0,10/review, taxa de bugs 23%
- **Diff + arquivos completos:** custo USD 2,50/review, taxa de bugs 14%, custo mensal USD 900
- **Code Review Graph:** custo USD 0,15/review, taxa de bugs 6%, custo mensal USD 54

Redução de 94% no custo, aumento de 62% na detecção de bugs.

Armadilhas a Evitar

- **Profundidade alta demais.** Profundidade 3 ou 4 captura quase todos os nós, anulando a economia. Comece com 2.
- **Ignorar pesos das arestas.** Todos os tipos iguais = contexto ruidoso. Calibre empiricamente.
- **Não atualizar o grafo.** Refatorações grandes quebram dependências. Reconstrua a cada release significativa.

Capítulo 2: Instalação e Configuração

De Zero a Review Automática em 10 Minutos

No Capítulo 1, você viu como o Code Review Graph resolve o problema dos tokens. Agora é hora de colocar as mãos no código.

O sistema foi projetado para duas formas de instalação: via `pip` para uso rápido, ou via `.mcp.json` para integração com IDEs que suportam o Model Context Protocol. Em menos de 10 minutos, o sistema está operacional.

Arquitetura em Quatro Componentes

O Code Review Graph tem quatro peças:

1. **Parser de AST** — analisa o código e extrai a estrutura (funções, classes, imports, chamadas)
2. **Construtor de grafos** — recebe a saída do parser e monta o grafo em memória, persistido como JSON em disco
3. **Calculador de blast radius** — BFS com filtragem por tipo de aresta e decaimento de peso
4. **Formatador de contexto** — transforma os arquivos selecionados em contexto comprimido para o LLM

Dependências

O sistema requer Python 3.9+ e as seguintes bibliotecas:

- **networkx** — manipulação de grafos (BFS, métricas de centralidade)
- **tree-sitter** — parser AST incremental de alta performance
- **click** — interface de linha de comando
- **pyyaml** — parser de configuração YAML

- **rich** — saída formatada no terminal

Para integração MCP, adicione `mcp` e `uvicorn`.

Instalação via Pip

O caminho mais rápido:

```
pip install code-review-graph
code-review-graph --version

# Instalar parsers para linguagens adicionais
code-review-graph install --languages go rust typescript
```

Depois, inicie no repositório:

```
cd /caminho/para/seu/repositorio
code-review-graph init
```

O comando `init` cria: - `.code-review-graph/config.yaml` — configuração - `.code-review-graph/graph.json` — grafo vazio - `.git/hooks/pre-push` — hook de review - `.git/hooks/post-commit` — hook de atualização do grafo

O arquivo de configuração padrão:

```
review:
  depth: 2
  max_tokens: 8000
  min_weight: 0.3
  languages: [python, javascript, typescript]

weights:
  call: 1.0
  import: 0.5
  inherit: 0.8
  use_data: 0.7
  comment: 0.1

output:
  format: markdown
  include_signatures: true
  include_docstrings: true

hooks:
  pre_push: true
```

```
post_commit: true
debounce_ms: 500

mcp:
  enabled: true
  port: 8472
```

Configuração Manual via .mcp.json

Para equipes que precisam de controle total, o arquivo `.mcp.json` na raiz do repositório:

```
{
  "mcpServers": {
    "code-review-graph": {
      "command": "code-review-graph",
      "args": ["serve", "--mcp"],
      "env": {
        "CRG_DEPTH": "2",
        "CRG_MAX_TOKENS": "8000",
        "CRG_WEIGHTS_CALL": "1.0",
        "CRG_WEIGHTS_IMPORT": "0.5",
        "CRG_OUTPUT_FORMAT": "markdown"
      }
    }
  }
}
```

Isso expõe o Code Review Graph como servidor MCP para Claude Code, Cursor, Windsurf ou VS Code.

Para sobrescrever configurações individuais, crie `.mcp.local.json` (adicionado ao `.gitignore`):

```
{
  "mcpServers": {
    "code-review-graph": {
      "env": {
        "CRG_DEPTH": "3",
        "CRG_MAX_TOKENS": "12000"
      }
    }
  }
}
```

Git Hooks: Reviews Automáticas

O hook `pre-push` dispara reviews antes de cada push:

```
cat > .git/hooks/pre-push << 'EOF'
#!/bin/bash
CHANGED_FILES=$(git diff --name-only origin/main...HEAD)
if [ -z "$CHANGED_FILES" ]; then exit 0; fi

echo "Code Review Graph: Analisando arquivos alterados..."
code-review-graph review \
  --files "$CHANGED_FILES" \
  --output comments \
  --format github

exit 0
EOF
chmod +x .git/hooks/pre-push
```

Para equipes com Husky (JavaScript/TypeScript):

```
{
  "lint-staged": {
    "*. {js,ts,py,go}": [
      "code-review-graph review --staged --format inline"
    ]
  }
}
```

Watch Mode: Feedback em Tempo Real

O watch mode monitora mudanças no repositório e gera reviews prévias em tempo real:

```
code-review-graph watch \
  --depth 2 \
  --max-tokens 8000 \
  --ignore "test/**" \
  --ignore "docs/**" \
  --notify terminal
```

Para rodar em segundo plano como daemon:

```
code-review-graph watch --daemon --pid-file /tmp/crg.pid
code-review-graph status
code-review-graph stop --pid-file /tmp/crg.pid
```

Construção do Grafo Inicial

Antes de usar pela primeira vez, construa o grafo:

```
code-review-graph build --recursive .

# Apenas linguagens específicas
code-review-graph build --languages python,javascript .

# Exportar para visualização
code-review-graph export --format dot --output graph.dot
dot -Tpng graph.dot -o graph.png
```

A primeira execução pode levar alguns minutos em repositórios grandes. Atualizações incrementais são rápidas.

Integração com CI/CD

GitHub Actions:

```
name: Code Review Graph
on:
  pull_request:
    types: [opened, synchronize, reopened]

permissions:
  contents: read
  pull-requests: write

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: actions/setup-python@v5
```

```

with: { python-version: '3.12' }
- run: pip install code-review-graph
- run: code-review-graph build --recursive .
- run: |
    code-review-graph review \
      --base origin/main --head HEAD \
      --output github-actions --format markdown
env:
  GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

```

GitLab CI:

```

code-review:
  image: python:3.12-slim
  stage: review
  before_script:
    - pip install code-review-graph
    - code-review-graph build --recursive .
  script:
    - |
      code-review-graph review \
        --base origin/main --head HEAD \
        --output gitlab-mr --format markdown
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

```

Verificação e Diagnóstico

Após a instalação, verifique tudo:

```

code-review-graph doctor

# Saída esperada:
# [OK] Python 3.12.4
# [OK] networkx 3.2.1
# [OK] tree-sitter 0.22.0
# [OK] Grafo construído (1.247 nós, 4.891 arestas)
# [OK] Git hook pre-push instalado
# [OK] Servidor MCP na porta 8472

```

Para problemas:


```
code-review-graph debug --verbose
code-review-graph review --file src/main.py --depth 2
code-review-graph graph --stats
code-review-graph cache --clear
```

Casos Reais

Startup de 20 devs: tempo de review caiu de 45 para 12 minutos. Taxa de bugs: 18% para 7%. Dois devs dedicados a reviews foram realocados para features.

Projeto open source: first response de 14 dias para 2 horas. Taxa de aceitação de PRs de 34% para 67%.

Empresa regulamentada: zero não conformidades em auditoria. Documentação de reviews automatizada, trabalho manual reduzido em 80%.

Erros Comuns na Instalação

- **Python incompatível:** use `python3 --version` e `pip3 install` se necessário
 - **Permissões de hooks:** `chmod +x .git/hooks/pre-push .git/hooks/post-commit`
 - **Cache desatualizado:** `code-review-graph cache --clear && build --recursive .`
 - **Conflito com Husky:** integre ao framework existente em vez de usar hooks separados
 - **Servidor MCP não inicia:** verifique porta livre com `code-review-graph doctor`
-

Próximos Passos

Este ebook cobriu os fundamentos: por que grafos resolvem o problema dos tokens e como instalar o Code Review Graph.

No livro completo, você encontra:

- **Personalização avançada** de pesos, profundidade e formatação de saída
 - **Suporte a múltiplas linguagens** com plugins de AST
 - **Padrões de projeto** para grafos de dependência em microserviços
 - **Métricas detalhadas** de qualidade e performance
 - **Integração avançada** com ferramentas de CI/CD e IDEs
 - **Estudos de caso** em organizações de diferentes portes
-

Quer ir além?

O livro completo *Code Review Graph: Redução de Contexto para Revisão de Código com IA* está disponível com todos os capítulos, incluindo implementação completa, arquitetura detalhada e casos de uso avançados.

O Code Review Graph não é apenas uma ferramenta — é uma abordagem que muda como sua equipe faz code review. Com ele, cada revisão é mais barata, mais precisa e mais rápida.

Para adquirir o livro completo e dominar todas as técnicas, acesse a Editora Agêntica.

Code Review Graph: Fundamentos — Ebook derivado do livro Code Review Graph: Redução de Contexto para Revisão de Código com IA

Copyright 2026 Heverton Eduardo Peres. Todos os direitos reservados.

Editora Agêntica

